

Task Offloading and Resource Scheduling in Mobile Edge-Cloud Computing Based on Edge Competition and Task Prediction

Shujuan Tian, Keke Xu, Shuhuan Xiang, Xingxia Dai, Zhu Xiao, *Senior Member, IEEE*, Li Zeng

Abstract—In the emerging cloud-edge-end computing networks, edge servers possess more constrained resources and face greater task offloading pressure than centralized cloud servers due to the surge in mobile applications and data. Concurrently, the presence of multiple edge service providers introduces additional challenges, including competition among servers, disordered resource pricing, and a lack of coordination in edge and cloud resource allocation. To address these issues, we propose a novel approach aimed at optimizing task deployment, resource pricing, and system coordination. First, we develop a competitiveness model to facilitate efficient edge-side task allocation while addressing the challenges of resource pricing under competitive conditions. Second, we design a transformer-based task prediction model to enhance the accuracy of resource demand forecasting, thereby enabling more effective edge-cloud resource allocation. To achieve these objectives, the system's interaction is structured into two distinct stages. This division simplifies the problem-solving process and ensures that the long-term goal of maximizing benefits for all stakeholders—edge service providers, cloud providers, and end-users—is achieved. The proposed solution not only improves task offloading efficiency and resource utilization, but also promotes fair competition and pricing transparency across the system.

Index Terms—Edge-Cloud computing, task offloading and prediction, resource pricing, resource scheduling.

I. INTRODUCTION

With the rapid proliferation of IoT devices such as smart-phones, wearable devices, uncrewed aerial vehicles (UAVs), and smart vehicles, the volume of data generated by these devices is growing exponentially [1]. However, the limited computing resources and storage capacity of IoT devices cannot adequately support the increasing demand for computational power and low-latency network services driven

by this surge in data traffic. Consequently, many computation-intensive tasks are offloaded to cloud servers [2], [3]. Yet, the significant delays associated with data transfer and task offloading to the cloud make it unsuitable for delay-sensitive tasks.

Edge computing, as a distributed architecture positioned closer to IoT devices, effectively addresses the latency challenges of delay-sensitive applications [4]. In multi-user systems, however, users often act selfishly, competing for shared computing resources. When resources are limited, such competition can lead to interference, reduced energy efficiency of edge servers, and resource underutilization. Standardized resource allocation methods may fail to meet diverse task requirements, resulting in unmet demand for some tasks while simultaneously wasting resources on others.

To address these issues, expanding the computing capabilities of edge servers has proven to be a viable approach to handling both delay-sensitive and computation-intensive tasks. While cloud servers possess substantial resources, efficiently migrating these resources to the edge to improve real-time network performance remains a significant challenge. Moreover, task offloading strategies are influenced by multiple factors, including user preferences, wireless communication conditions, offloading benefits, and the availability of edge and cloud resources [5]. For IoT devices, the offloading strategy involves cloud and edge servers with different pricing models, so container orchestration and other methods of employing cloud resources to augment edge-side capabilities influence the task offloading decision. One of the key optimization goals for task offloading is to maximize user benefit while also considering latency and the Quality of Service (QoS), which are among the primary motivations for offloading tasks.

We introduce a competitiveness model designed to optimize task allocation for edge servers and establish a fair, competitive resource pricing mechanism. This model accounts for the varying capabilities and costs of edge servers, as well as the specific requirements of tasks, ensuring equitable resource distribution while maximizing utilization efficiency. Servers closer to users are assigned higher weights, as reduced latency enhances the users' QoS.

Building on this foundation, we develop a Transformer-based task prediction model, leveraging deep learning to forecast task demands and optimize real-time resource allocation in edge-cloud environments. By accurately predicting task arrivals and resource needs, this model facilitates efficient resource scheduling, significantly enhancing the overall perfor-

This work was supported in part by National Natural Science Foundation of China under Grant No. 62172349, U24A20247 and 62272152; in part by Major landmark innovation demonstration project in Hunan Province No. 2022XK2301; in part by National Natural Science Foundation of Hunan Province under Grant No. 2023JJ30597 and 2025JJ50351; in part by Shenzhen Science and Technology Program under Grant No. JCYJ20240813162405007. (Corresponding author: Xingxia Dai)

Shujuan Tian, Keke Xu, Shuhuan Xiang, Xingxia Dai and Li Zeng are with the School of Computer Science, Xiangtan University, Xiangtan, 411105, China (e-mail: sjtianwork@xtu.edu.cn; 202221633058@smail.xtu.edu.cn; 202321633040@smail.xtu.edu.cn; xingxdai@xtu.edu.cn; xtuzengli@163.com).

Zhu Xiao is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, 410082, and also with Shenzhen Research Institute of Hunan University, Shenzhen, 518055 China (e-mail: zhuxiao@hnu.edu.cn).

mance and responsiveness of the computing network. To address scenarios in which multiple edge servers simultaneously request cloud resources, we employ a multi-round Vickrey auction mechanism to determine both resource pricing and allocation, ensuring fairness and efficiency.

Our solution adopts a two-phase system interaction approach. The first phase focuses on task allocation and resource pricing, laying the groundwork for efficient operations. The second phase dynamically optimizes resource allocation using insights from the task prediction model, aligning real-time adjustments with future demands. This two-phase framework strikes a balance between immediate operational efficiency and long-term strategic optimization, ultimately maximizing benefits for all stakeholders in the computing network.

The contributions of this paper mainly include the following three aspects:

- Introduced a two-phase system interaction approach to balance short-term operational efficiency and long-term strategic optimization. The first phase focuses on task allocation and resource pricing, while the second phase dynamically optimizes resource allocation based on predictions from the task prediction model. This approach maximizes the benefits for all stakeholders in the computing network.
- Designed a competitiveness model to optimize task allocation among edge servers and establish a fair and competitive resource pricing mechanism. The model considers the capabilities and costs of different edge servers, as well as task-specific requirements. It also assigns higher weights to servers closer to users, thereby enhancing resource utilization efficiency and improving QoS.
- Developed a Transformer-based task prediction model leveraging deep learning to accurately forecast task arrivals and resource demands. This model enables real-time optimization of edge-cloud resource allocation, improving the overall performance and responsiveness of the computing network. Additionally, a multi-round Vickrey auction mechanism was employed to achieve efficient resource pricing and allocation when multiple edge servers compete for cloud resources.

The structure of the paper is organized as follows. Section II reviews related work. In Section III, we present the system model and formulate the offloading and pricing challenges as multi-objective optimization problems. Section IV outlines the proposed algorithm to address these problems. In Section V, we conduct extensive experiments and comparisons to validate the feasibility of the proposed solutions. Finally, Section VI summarizes the contributions of this paper.

II. RELATED WORK

With the increasing data generated by IoT devices, task offloading has become a critical solution to meet the demands for high computational power and real-time performance [6]. Extensive research has addressed offloading problems in cloud computing, which can be categorized into single-cloud [7] and multi-cloud offloading [8]. Common approaches to generating offloading decisions include machine learning [9], game theory

[10], and genetic algorithms [11]. Moreover, efforts have been made to optimize cloud architectures [12], [13]. Similarly, resource allocation and task offloading have been widely studied in edge computing. Edge computing tasks are typically offloaded partially or entirely, employing methods such as deep reinforcement learning [5], [14], [15], evolutionary algorithms [16], [17], and traditional optimization techniques [18], [19]. Task offloading in edge-cloud cooperative environments focuses on objectives like minimizing delay [20], energy consumption [21], and costs [22], as well as achieving load balancing [23] and other performance improvements [24].

Deep learning-based task prediction and offloading strategies have been widely studied to optimize MEC performance. Miao et al. proposed an LSTM-based task prediction algorithm to enhance task migration, reducing system delay under dynamic workloads [25]. Wu et al. developed an energy-efficient task offloading mechanism for UAV-enabled MEC networks, improving resource utilization and energy efficiency [26]. Chen et al. introduced a context-aware task scheduling approach for time-series prediction in cloud-edge architectures, enabling efficient data localization and processing across edge nodes [27]. Additionally, Xiao et al. proposed a two-stage simulation task partitioning method using ensemble learning, achieving a 26% runtime reduction by optimizing resource allocation and scheduling for dynamic workloads in cloud environments [28]. These studies underscore the potential of advanced learning techniques in enhancing task prediction, resource allocation, and overall efficiency in MEC and cloud-edge systems.

Additionally, pricing mechanisms for edge-cloud offloading have gained considerable attention. Duan *et al.* [29] formulated an optimization model that minimizes offloading costs while ensuring queue stability. By constructing a potential game framework for the edge-cloud environment, individual users could selfishly minimize their costs while optimizing computing offloading strategies [30]. Other studies analyzed the interactions between devices and edge/cloud servers, enabling users to adopt game-theoretic approaches for task pricing and offloading decisions [31]–[37].

Unlike previous studies, this work divides task offloading into two distinct phases. In the first phase, a resource pricing mechanism is employed to determine task allocation, prioritizing both latency-sensitive tasks and user QoS. Specifically, tasks are offloaded to edge servers that not only meet base station (BS) coverage requirements but are also optimally located to minimize user latency, thereby enhancing QoS. In the second phase, a Transformer-based prediction model is used to forecast upcoming tasks for each edge server. By leveraging cloud-edge orchestration techniques such as container orchestration, the predicted tasks are utilized to optimize resource scheduling, improving system load management and achieving long-term performance benefits for the edge-cloud environment.

III. SYSTEM MODEL

A. Scenery and Modelling

In this section, the physical scenario of the system model is described in detail. IoT is ubiquitous in our daily life, and the

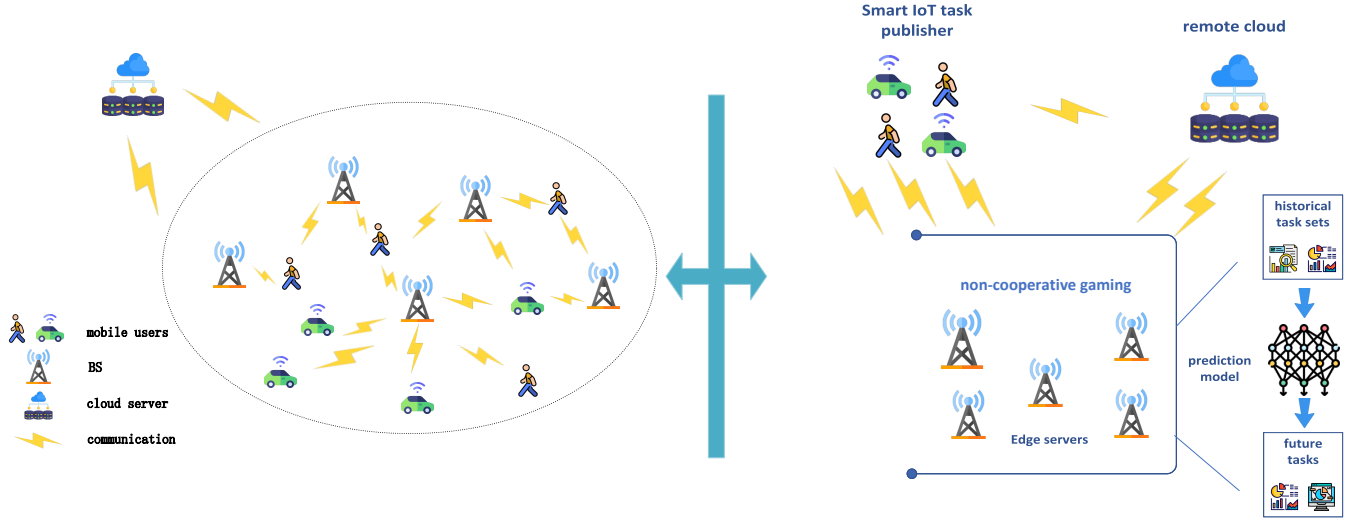


Fig. 1: System framework and data flow

explosive growth of data leads to an increasing demand for computing power by mobile users. As a result, users need to offload their tasks to servers with higher computing or processing power. Three entities are involved in this process: the user mobile device, the edge server and the remote cloud server. The cloud servers are located far away from the user's device and provide storage computing services to the remote users through the core network and are connected to the edge servers through the wired core network. Due to the high latency caused by the distance limitation, the latency-sensitive users will offload their tasks to the closer edge servers. However, since the cloud servers have sufficient resources and the edge computing resources are scarce, the cloud servers can respond to the resource demands of the edge servers in the form of container orchestration. When the load on the edge servers is high, computing resources can be rented from the cloud to ensure the quality of user service experience.

Edge servers are connected to cellular BSs through wireless networks. These BSs have different coverage radius ranges, and user devices located in the overlapping areas of multiple BSs can simultaneously send task offload requests to the edge servers connected to these BSs. Since the edge servers may come from different providers, they are in a competitive relationship. It should be noted that to ensure service continuity and to take into account the competitive relationship of edge servers, the user will offload tasks only to the edge servers that have been in the service range during the task execution. When resources are insufficient edge servers will only rent resources from the cloud and not from other edge servers with idle resources.

For example, in the Central Business District (CBD) of a city with a high density of mobile users, pedestrians may have latency-insensitive tasks, such as a mobile health monitoring application in which the user collects physiological data, including heart rate, step count, sleep quality, etc., through wearable devices (e.g., smartwatches, fitness trackers). These

data are initially processed on the cell phone side, and can be regularly uploaded to the server for more in-depth analysis and storage, as the real-time requirement is not high, which can be offloaded to the cloud to save costs. Whereas mobile vehicles may have low latency requirements real-time processing tasks will be offloaded to edge servers with lower latency. When there are a lot of vehicles and the edge servers are under a lot of pressure, it is possible to apply for rented resources from cloud servers far away from the city. In this process, pedestrian or moving vehicle users will only offload tasks to servers within the coverage area. If the user is within the coverage of multiple servers, distributed offloading improves efficiency and reduces server pressure. However, tasks will only be offloaded to one of the cloud or edge servers based on latency sensitivity.

The system framework between user devices, edge servers and cloud servers is shown in Fig.1. In this framework, the user device sends the offload request to the cloud server or the edge server within its BS coverage based on its location. When a user offloads a task to multiple edge servers, the edge servers adjust their competitiveness based on our competitiveness model to complete the competitive task deployment process until it finally converges to the optimal allocation. Each time the task is completed, the edge server will evaluate its own resource status through the prediction model, and rent resources from the cloud when the pressure is too high. In order to ensure continuous service capability. The users and edge servers are represented by sets $\mathcal{U} = \{u_1, u_2, \dots, u_N\}$ and $\mathcal{E} = \{e_1, e_2, \dots, e_M\}$ respectively. Each edge server records its own data for each task performed for analysis and prediction. We use $\mathcal{H}_m = \{(t_1, r_1, e_1), (t_2, r_2, e_2), \dots, (t_I, r_I, e_I)\}$ to denote the set of historical tasks for e_m . Where t_I denotes the timestamp of the task, r_I denotes the amount of resources used for the task, and e_I denotes the execution time of the task.

TABLE I: System Parameters

Parameter	Description
$\mathcal{U} = \{u_1, u_2, \dots, u_N\}$	Set of users
$E = \{e_1, e_2, \dots, e_M\}$	Set of edge servers
$\mathcal{H}_m = \{(t_1, r_1, e_1), (t_2, r_2, e_2), \dots, (t_I, r_I, e_I)\}$	Historical task set of edge server e_m
$S_{n,m}$	Transmission rate between user u_n and edge server e_m
$S_{n,cloud}$	Transmission rate between user u_n and cloud server
B_m^n	Bandwidth between user u_n and edge server e_m
p_n	The transmit power of the user u_n
$A_{n,m}$	the level of competitiveness of the server e_m in the user u_n 's task
$r_{n,m}$	Resource requirements for user u_n to execute tasks on edge server e_m
$H_{n,m}$	Channel gain between user u_n and edge server e_m
σ^2	Background noise power of edge servers
f_m	Clock frequency of edge server e_m
f_{cloud}	Clock frequency of the central cloud
$T_{n,m}$	Total time of task offloaded to edge server e_m
$T_{n,cloud}$	Total time of task offloaded to the cloud server
$t_{n,cloud}^{exe}$	Execution time of task offloaded to the cloud server
$t_{n,m}^{exe}$	Execution time of task offloaded to the edge server
$t_{n,m}^{tran}$	Transmission time of task offloaded to the edge server
$t_{n,cloud}^{tran}$	Transmission time of task offloaded to the cloud server
D_n	Task packet size of user u_n
C_n	CPU requirement of task u_n
x	Price coefficient
y	Distance coefficient
a	Price factor
b	Transmission time factor
c	Execution time factor
$p_{n,m}$	Price for offloading user u_n 's task to edge server e_m
$\bar{p}_{n,m}$	The average price of historical task deployments on the edge server e_m
$p_{n,cloud}$	Price for offloading user u_n 's task to the cloud server
$q_{m,cloud}$	The number of resources requested by edge server e_m from the cloud server
$d_{n,m}$	Distance between user u_n and edge server e_m
$QoS_{n,m}$	Quality of Service (QoS) for user u_n offloading to e_m
$Q_{n,m}$	QoS weight of the edge server
a_n	Benefit coefficient of user u_n
R_n	Resource demand of user u_n 's task
R_{cloud}	The number of the cloud server resources
W	System incentive
h_m	Unit resource cost of edge server e_m
h_{cloud}	Unit resource cost of cloud server
E_M	Edge server set
res_M	Amount of resources remaining on edge servers
h_M	Unit cost set of edge servers
L_M	Location set of edge servers
L_N	Location of user u_n
Dir_n	Direction of user u_n
v_n	Speed of user u_n
ϵ	Control parameter
T	Time horizon

B. Latency Model

The latency model mainly considers offloading time, including transmission latency and task execution time. Devices can offload tasks to edge servers or cloud servers as required. Since the size of the task result is much smaller than the size of the task, and the downstream speed of transmission is much larger than the upstream speed, we ignore the time for the task to be transmitted from the cloud to the edge node and the time for the task result to be transmitted from the edge node to the user's device [31]. Thus the user's task offloading delay is divided into two cases. One is that the user unoffloads the task to the edge server e_m , then the delay $T_{n,m}$ includes the transmission time of the task package from the user device to the edge server $t_{n,m}^{tran}$ and the execution time of the task at the edge server $t_{n,m}^{exe}$; the second is that the user offloads the task to the cloud server, then the delay $T_{n,cloud}$ includes the transmission time of the task package from the user device

to the edge server $t_{n,m}^{tran}$ and then to the cloud server, the transmission time $t_{n,cloud}^{tran}$ and the execution time of the task at the cloud server $t_{n,cloud}^{exe}$. The data transfer rate we calculate using Shannon's formula [20]:

$$S_{n,m} = B_m^n \log_2(1 + \frac{p_n H_{n,m}}{\sigma^2}), \quad (1)$$

where p_n denotes the transmit power of the user u_n , usually measured in watts. Increasing the transmit power increases the signal strength when it reaches the receiving end, thus potentially increasing the data transfer rate. The following latency can then be derived for the two offloading methods:

$$T_{n,m} = t_{n,m}^{tran} + t_{n,m}^{exe} = \frac{D_n}{S_{n,m}} + \frac{C_n}{f_m}, \quad (2)$$

$$\begin{aligned} T_{n,cloud} &= t_{n,m}^{tran} + t_{n,cloud}^{tran} + t_{n,cloud}^{exe} \\ &= \frac{D_n}{S_{n,m}} + \frac{D_n}{S_{n,cloud}} + \frac{C_n}{f_{cloud}}, \end{aligned} \quad (3)$$

where $S_{n,m}$ and $S_{n,cloud}$ represent the data rate of task u_n offloaded to edge server e_m and the cloud server respectively.

The system parameters are shown in the TABLE I.

C. Competitiveness and QoS Model

1) *Competitiveness Model*: When a user u_n offloads a task to an edge server in the coverage area, the edge servers compete for the right to deploy the task. Since different edge servers e_m have different distances from users, i.e., they have different $d_{n,m}$ so they have different delays $T_{n,m}$, the distance directly determines the convenience of providing services to users, so we attach great importance to the impact of distance on the competitiveness of edge servers. In order to provide users with a basis for selecting different edge servers, we design a competitiveness model based on the price of the server service $p_{n,m}$ and the distance from the user $d_{n,m}$. The details are as follows:

$$A_{n,m} = \frac{\frac{x}{p_{n,m}} + \frac{y}{d_{n,m}}}{\sum_{m=1}^M (\frac{x}{p_{n,m}} + \frac{y}{d_{n,m}})} \quad (4)$$

$A_{n,m}$ reacts to the level of competitiveness of the server e_m in this user u_n 's task. Where x and y are the price and distance coefficients, respectively. Edge servers adjust their competitiveness to win the competition by adjusting their price based on real-time distance. Edge servers with shorter distances and lower prices will be more competitive.

2) *QoS Model*: We use $QoS_{n,m}$ to evaluate the user experience generated by edge servers that complete user task offloads. Due to different prices and latencies, users have different QoS for different edge servers' services. we propose to evaluate user QoS using edge servers' price $p_{n,m}$, transmission latency $t_{n,m}^{tran}$, and computation latency $t_{n,m}^{exe}$ as follows:

$$QoS_{n,m} = \frac{a}{p_{n,m}} + \frac{b}{t_{n,m}^{tran}} + \frac{c}{t_{n,m}^{exe}} \quad (5)$$

where a , b and c are the price, transmission latency and computation latency coefficients, respectively. The larger the $QoS_{n,m}$, the higher the quality of the user experience.

D. Benefits Model

This section describes the benefits modelling and objectives of the three types of entities in the system. The interaction process of the system is that the user u_n sends an offloading demand to the edge server or cloud server in the range, and the amount of resources required is R_n . If offloaded to the edge, the edge servers adjust the competitiveness of each other $A_{n,m}$ competing for the right to deploy the task. After completing the task deployment, the edge servers decide whether to request resources from the cloud servers based on their load conditions; if it is a latency-insensitive task it is offloaded to the cloud for execution.

1) *Benefits for Users* : The user revenue consists of two parts, one from the reward revenue of task completion and the other is the cost of deploying the task. Users can choose to offload tasks to the edge or the cloud based on the servers that satisfy their latency and QoS requirements. The revenue is expressed as follows:

$$\begin{aligned} \Phi_{user} &= k_{n,m} \left(\sum_{m=1}^M a_n r_{n,m} t_{n,m}^{exe} - \sum_{m=1}^M p_{n,m} r_{n,m} t_{n,m}^{exe} \right) \\ &+ k_{n,cloud} (a_n R_n t_{n,cloud}^{exe} - p_{n,cloud} R_n t_{n,cloud}^{exe}) \end{aligned} \quad (6)$$

The user's objectives are as follows:

$$\begin{aligned} &\max_{k_{n,r,p}} \Phi_{user}, \\ &s.t. (8a), (8b), (8c), (8d), (8e). \end{aligned} \quad (7)$$

where $k_{n,m}$ and $k_{n,cloud}$ is the control parameter, $k_{n,m} + k_{n,cloud} = 1$, $k_{n,m} = 1$ indicates that the task is offloaded to the edge server, and $k_{n,m} = 0$ otherwise. The benefit factor is represented by a_n , while $t_{n,m}^{exe}$ and $t_{n,cloud}^{exe}$ denote the execution times of the task on the edge server and the cloud server, respectively. The resource requirements for task execution on the edge and cloud servers are given by $r_{n,m}$ and R_n , respectively. Additionally, $p_{n,m}$ and $p_{n,cloud}$ represent the unit prices of resources on the edge server and the cloud server, respectively.

There are some constraints in achieving the above goals. Constraint C1 ensures that the edge server allocates fewer servers to users than its own total resources. Constraint C2 specifies that the resources allocated by the cloud server to the user and the edge are less than the total resources, where R_{cloud} is the number of resources on the cloud server, and $q_{m,cloud}$ is the number of resources requested by edge server e_m from the cloud server. Constraints C3 and C4 state that tasks offloaded to the edge or cloud server satisfy the user's latency requirements. Constraint C5 ensures that the cloud server resources are cheaper than edge server resources.

$$C1 : \sum_{n=1}^N r_{n,m} \leq R_m, \forall m, \quad (8a)$$

$$C2 : \sum_{n=1}^N R_n + \sum_{m=1}^M q_{m,cloud} \leq R_{cloud}, \quad (8b)$$

$$C3 : T_{n,cloud} \leq T_n^{max}, \forall n, \quad (8c)$$

$$C4 : T_{n,m} \leq T_n^{max}, \forall n, m, \quad (8d)$$

$$C5 : p_{n,cloud} \leq p_{n,m}, \forall n, m. \quad (8e)$$

2) *Benefits for Edges* : The edge server revenue consists of three parts, one part comes from the system incentive reward for completing the task deployment, denoted as W the second part is the cost price difference of the computing resources sold to the user, and the third part is the cost of leasing the cloud server resources. The system incentive is determined based on the QoS provided by the edge server to the user, denoted as $Q_{n,m}W$. $Q_{n,m}$ represents the QoS weight of the edge server, as defined below:

$$Q_{n,m} = \frac{QoS_{n,m}}{\text{Max}\{QoS_{n,1}, QoS_{n,2}, \dots, QoS_{n,M}\}} \quad (9)$$

Users send offload requests like multiple servers, so the price $p_{n,m}$ of servers in competition is dynamically adjusted in order to obtain more task assignments. $A_{n,m}$ calculated from Eq. (4). Thus $r_{n,m}$ represents the amount of resources required for the task assigned by edge server e_m .

$$r_{n,m} = A_{n,m}R_n \quad (10)$$

The benefit function is specifically defined as follows:

$$\Phi_{edge} = Q_{n,m}W + t_{n,m}^{exe}r_{n,m}(p_{n,m} - h_m) - k_m t_{m,cloud}^{exe}p_{m,cloud}q_{m,cloud} \quad (11)$$

The edge's objectives are as follows:

$$\begin{aligned} & \max_{k_m, Q, r, p} \Phi_{edge}, \\ & \text{s.t. (13a), (13b), (13c), (13d), (13e), (13f)}. \end{aligned} \quad (12)$$

where k_m is a control parameter, the edge server will decide whether to lease resources from the cloud server based on the real-time load to better deploy future tasks and maximise the long-term benefits, if it leases resources from the cloud server then $k_m = 1$, otherwise it is 0. Where $t_{n,m}^{exe}$ is the execution time of the task at the edge server e_m , h_m is the resource unit cost of the edge server e_m .

There are some constraints in achieving the above goals. Constraint C1 ensures that the edge server allocates fewer servers to users than its own total resources. Constraint C2 states that the edge price is greater than the cost. Constraints C3 indicate that the cloud price is greater than the cost. Constraints C4 state that tasks offloaded to the edge server satisfy the user's latency requirements. Constraint C5 ensures that the cloud server resources are cheaper than edge server resources. Constraint C6 ensures that the auction price of edge servers, i.e., the price of requesting resources from the cloud, shall not exceed the average price of their historical task deployments, where $\bar{p}_{n,m}$ is the average price of historical task deployments on the edge server e_m .

$$C1: \sum_{n=1}^N r_{n,m} \leq R_m, \forall m, \quad (13a)$$

$$C2: h_m \leq p_{n,m}, \forall n, m, \quad (13b)$$

$$C3: h_{cloud} \leq p_{m,cloud}, \forall m, \quad (13c)$$

$$C4: T_{n,m} \leq T_n^{max}, \forall n, m, \quad (13d)$$

$$C5: p_{n,cloud} \leq p_{n,m}, \forall n, m, \quad (13e)$$

$$C6: p_{m,cloud} \leq \bar{p}_{n,m}, \forall n, m. \quad (13f)$$

3) *Benefits for Cloud* : The revenue from cloud servers is similarly divided into two parts, one from deploying user

latency-insensitive tasks and the other from leasing computing resources to edge servers, as defined below:

$$\begin{aligned} \Phi_{cloud} &= t_{n,cloud}^{exe}R_n(p_{n,cloud} - h_{cloud}) \\ &+ k_m \sum_{m=1}^M t_{m,cloud}^{exe}(p_{m,cloud} - h_{cloud})q_{m,cloud} \end{aligned} \quad (14)$$

The cloud's objectives are as follows:

$$\begin{aligned} & \max_{p_{n,cloud}, p_{m,cloud}} \Phi_{cloud}, \\ & \text{s.t. (16a), (16b), (16c), (16d), (16e), (16f)}. \end{aligned} \quad (15)$$

where $t_{n,cloud}^{exe}$ is the execution time of the task on the cloud server, h_{cloud} is the unit cost of cloud resources, and $q_{m,cloud}$ is the amount of resources requested for the edge server.

There are some constraints in achieving the above goals. Constraint C1 specifies that the resources allocated by the cloud server to the user and the edge are less than the total resources. Constraints C2 and C3 indicate that the cloud price is greater than the cost. Constraints C4 state that tasks offloaded to the cloud server satisfy the user's latency requirements. Constraint C5 ensures that the cloud server resources are cheaper than edge server resources. Constraint C6 ensures that the auction price of edge servers, i.e., the price of requesting resources from the cloud, shall not exceed the average price of their historical task deployments.

$$C1: \sum_{n=1}^N R_n + \sum_{m=1}^M q_{m,cloud} \leq R_{cloud}, \quad (16a)$$

$$C2: h_{cloud} \leq p_{n,cloud}, \forall n, \quad (16b)$$

$$C3: h_{cloud} \leq p_{m,cloud}, \forall m, \quad (16c)$$

$$C4: T_{n,cloud} \leq T_n^{max}, \forall n \quad (16d)$$

$$C5: p_{n,cloud} \leq p_{n,m}, \forall n, m, \quad (16e)$$

$$C6: p_{m,cloud} \leq \bar{p}_{n,m}, \forall n, m. \quad (16f)$$

During dynamic system interactions, the cloud server, edge servers, and users each aim to maximize their own benefits, as shown in Eq. (7), Eq. (12), and Eq. (15), while being mutually constrained. Between users and edge servers, both are affected by the price $p_{n,m}$: users prefer lower prices, and edge servers prefer higher ones. Due to market competition, edge servers adjust prices to remain competitive, leading to a reasonable equilibrium. Between edge servers and the cloud server, the price $p_{m,cloud}$ influences both sides. Edge servers use auctions to lease cloud resources, but avoid prices higher than the historical average price of their task deployments to prevent losses. Between users and the cloud server, the price $p_{n,cloud}$ also matters. If it's higher than that of edge servers, users will offload tasks to the edge for lower cost and latency. To attract users, the cloud adjusts its pricing accordingly. Following our algorithmic strategy, all three parties determine optimal pricing to maximize their own gains. Moreover, our task prediction and resource pre-reservation mechanisms enhance edge server capacity, increasing task offloading and overall system utility.

IV. PROBLEM FORMULATION AND ALGORITHMS

The proposed system architecture consists of three hierarchical layers: cloud servers, distributed edge servers, and diverse IoT users. It operates in two phases. First, IoT users offload tasks to cloud or edge servers based on latency, resource needs, and QoS. Second, edge servers predict workloads and request cloud resources to optimize allocation. We present structured algorithms for both phases, forming an integrated framework for efficient interaction in dynamic conditions.

A. Task Allocation and Resource Pricing

For latency-insensitive tasks, users prefer offloading to the cloud based on cost, with benefits computed using equations (6) and (14). For latency-sensitive tasks, users offload to nearby edge servers. The set of available edge servers, E' , varies with user location and time. Servers in E' interact using the competitiveness model (4) and update their competitiveness with the formula in equation (17).

$$p_{n,m}^t = \left(1 - \frac{\sum_{m=1}^M \frac{x}{p_{n,m}^{t-1}} + \frac{y}{d_{n,m}}}{\frac{xM}{p_{n,m}^{t-1}} + \frac{yM}{d_{n,m}}} \right) p_{n,m}^{t-1} + p_{n,m}^{t-1} \quad (17)$$

When the results stabilise, a set of equilibrium solutions will be produced. The following is the flow of the algorithm:

Algorithm 1 Balanced Competitiveness Task Allocation Algorithm

```

1: Input: Edge server set  $E_M$ , unit cost set  $h_M$ , locations set  $L_M$ , Amount of resources  $res_M$ , task pack  $D_n$  task resource  $R_n$ , user location  $L_n$ , user direction  $Dir_n$ , speed  $v_n$ , control parameter  $\epsilon$ 
2: Output: Server set  $E_t$ , prices  $P_t$ , task allocations  $R_t$ 
3: if  $T_{n,cloud} \leq T_n^{max}$  then
4:   Calculate user and cloud benefits using Eq. (6)(14)
5:   Return:  $\Phi_{users}$ ,  $\Phi_{cloud}$ 
6: end if
7: Compute task execution time  $T_{n,m}$  using Eq. (2)
8: Derive candidate server set  $E'$  based on  $L_n$ ,  $v_n$ ,  $res_M$ ,  $Dir_n$ ,  $T_{n,m}$ , and  $L_M$ 
9: for each server  $e_m \in E'$  do
10:   $e_m$  reports initial price  $p_m$ 
11:  Compute  $A_{n,m}$  using Eq. (4)
12: end for
13: while  $E' \neq \emptyset$  do
14:   for each server  $e_m \in E'$  do
15:    Adjust  $A_{n,m}$  using Eq. (17) and Eq. (4)
16:    if  $p_{n,m} < h_m$  ( $h_m \in H_M$ ) then
17:      Remove  $e_m$  from  $E'$ 
18:    end if
19:   end for
20:   if all  $e_m \in E'$  satisfy  $|p_{n,m}^{(t)} - p_{n,m}^{(t-1)}| < \epsilon$  then
21:     Record current prices  $P_t$ 
22:     Compute  $r_{n,m}$  for  $E'$  using Eq. (4) and Eq. (10)
23:     break
24:   end if
25: end while
26: Return:  $E_t = E'$ ,  $P_t = \{p_{n,m} \mid e_m \in E'\}$ ,  $R_t = \{r_{n,m} \mid e_m \in E'\}$ 

```

The time complexity of Algorithm 1 is related to the number of edge servers and the number of iterative rounds for price adjustment, which is still mainly affected by the number of edge servers, thus the time complexity is $O(M)$. It will complete the deployment of resource pricing and task allocation for offloading to edge servers.

B. Task Prediction

The remaining resources of an edge server is an important factor that often determines whether it can offload the user's tasks; at the same time, fewer resources mean higher prices, so edge servers with fewer remaining resources are more likely to drop out of the competition in the market. Using our prediction model with the distributed architecture of the edge cloud for resource pre-purchase can handle the problem well.

Edge servers make decisions about whether to pre-purchase resources by predicting future tasks and thus determining whether they will be able to participate in the offloading of tasks at some point in the future, determining whether their load capacity is insufficient, and ultimately completing the decision of whether to pre-purchase resources or not.

The output of Algorithm 1 is the set of servers for executing the tasks and the resources used for deploying the tasks. After the execution of Algorithm 1, the allocation of tasks and pricing of resources are completed, but when the tasks at the edge are too concentrated, there is the problem of busy at the edge and idle in the cloud. So the next problem that needs to be solved is resource allocation. Whenever a task is executed, the edge server predicts the upcoming tasks through the prediction model based on its own historical task records and historical task transaction prices, and when assessing the lack of resources, it will apply for resources from the cloud to increase its own load and improve the expected revenue by means of container orchestration, so as to achieve the maximisation of the long-term benefits, and also improve the revenue of the cloud server. The reason for choosing container orchestration technology is that containerization technology has become the mainstream of modern distributed application development and deployment, and container orchestration system further improves the efficiency of container management and scheduling. Its main advantages include: automated deployment and management, flexible scheduling and resource allocation, continuous delivery and version control, and cross-platform compatibility. Container orchestration technology can help developers split applications into multiple microservices to achieve flexible scheduling and elastic expansion. Intelligent scheduling of workloads between the edge and the cloud through the orchestration platform not only optimizes network latency, but also ensures that business logic runs efficiently and collaboratively among different nodes. A collection of historical task record data is passed into the transformer-based deep learning model for training, which can output a corresponding prediction data. The historical data contains the task timestamps, the amount of resources used, and the execution time, as do the predicted results. We use the generated data results as a reference for upcoming tasks to assess the adequacy of the edge server's resources at the moment and in the coming period.

Our design of the prediction model is divided into five areas: data processing, model definition, training, incremental training, and prediction. The details are as follows:

Given a task history dataset:

$$\mathcal{H} = \{(t_i, r_i, e_i)\}_{i=1}^I,$$

where t_i is the timestamp of task i , r_i is the resource usage of task i , and e_i is the execution time of task i , the input sequence is constructed as:

$$x_j = (r_j, e_j, \Delta t_j), \quad \Delta t_j = t_j - t_{j-1}, \quad \forall j \in \{2, \dots, I\}.$$

The target sequence is:

$$y_i = (\Delta t_{i+1}, r_{i+1}, e_{i+1}), \quad \Delta t_{i+1} = t_{i+1} - t_i.$$

The dataset for training is represented as:

$$\mathcal{D} = \{(X_i, y_i)\}, \quad X_i = \{x_1, x_2, \dots, x_i\}, \quad \forall i \in \{2, \dots, I\}.$$

The Transformer-based model includes an embedding layer, encoder, and output layer. The embedding layer maps $x_j \in \mathbb{R}^{d_{in}}$ to a higher-dimensional space via:

$$\text{Embedding}(x_j) = W_e \cdot x_j + b_e,$$

where $W_e \in \mathbb{R}^{d_{in} \times d_{model}}$. The Transformer encoder processes the input using L layers of multi-head attention and feedforward sub-layers:

$$H_l = \text{TransformerLayer}(H_{l-1}), \quad l = 1, \dots, L,$$

$$H_0 = \text{Embedding}(X).$$

Attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

and the feedforward network is defined as:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

The output layer predicts the target:

$$\hat{y} = W_o \cdot h_L + b_o, \quad h_L \in \mathbb{R}^{d_{model}}, \quad W_o \in \mathbb{R}^{d_{model} \times d_{out}}.$$

The loss function for training is:

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^M \|\hat{y}_i - y_i\|_2^2.$$

The training process involves computing embeddings, passing them through the Transformer encoder, predicting outputs, and updating model parameters via gradient descent:

$$\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}.$$

For incremental training with new data $\mathcal{H}_{\text{new}} = \{(t_i, r_i, e_i)\}_{i=I+1}^{I+K}$, incremental data $\mathcal{D}_{\text{new}}$ is generated, and the model is fine-tuned with early stopping to prevent overfitting:

$$\text{If } \frac{1}{\tau} \sum_{e=i}^{i+\tau} \mathcal{L}_e > \mathcal{L}_{\min}, \text{ stop training.}$$

For prediction, given a task history $\mathcal{H} = \{(t_i, r_i, e_i)\}_{i=1}^I$, the input sequence $X_I = \{x_1, x_2, \dots, x_I\}$ is constructed and passed through the model to predict:

$$\hat{y}_I = (\Delta \hat{t}_{I+1}, \hat{r}_{I+1}, \hat{e}_{I+1}),$$

where the next task timestamp is computed as:

$$\hat{t}_{I+1} = t_I + \Delta \hat{t}_{I+1}.$$

The predicted next task is:

$$\hat{T} = (\hat{t}_{I+1}, \hat{r}_{I+1}, \hat{e}_{I+1}).$$

The specific algorithm is as follows:

Algorithm 2 Transformer-Based Task Prediction

- 1: **Input:** Task history $\mathcal{H} = \{(t_i, r_i, e_i)\}_{i=1}^I$, model parameters Θ , learning rate η , number of epochs E
 - 2: **Output:** Predicted next task $\hat{T} = (\hat{t}_{I+1}, \hat{r}_{I+1}, \hat{e}_{I+1})$
 - 3: Compute input sequence $X = \{x_1, \dots, x_{I-1}\}$, where $x_j = (r_j, e_j, \Delta t_j)$, and $\Delta t_j = t_j - t_{j-1}$
 - 4: Compute target sequence $Y = \{y_2, \dots, y_I\}$, where $y_i = (\Delta t_{i+1}, r_{i+1}, e_{i+1})$
 - 5: Initialize Transformer model $\mathcal{T}$ with parameters Θ
 - 6: **for** epoch = 1 to E **do**
 - 7: $\mathcal{L} \leftarrow 0$
 - 8: **for** each training sample (X_i, y_i) in $\mathcal{D}$ **do**
 - 9: $H_0 \leftarrow \text{Embedding}(X_i)$
 - 10: $H_L \leftarrow \mathcal{T}(H_0)$
 - 11: $\hat{y}_i \leftarrow \text{OutputLayer}(H_L[-1])$
 - 12: $\mathcal{L}_i \leftarrow \text{MSE}(\hat{y}_i, y_i)$
 - 13: Update $\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}_i$
 - 14: $\mathcal{L} \leftarrow \mathcal{L} + \mathcal{L}_i$
 - 15: **end for**
 - 16: **Early Stopping Check:**
 - 17: **if** Validation loss increases for consecutive epochs **then**
 - 18: **Break**
 - 19: **end if**
 - 20: **end for**
 - 21: Construct input $X_I = \{x_1, x_2, \dots, x_I\}$
 - 22: $H_0 \leftarrow \text{Embedding}(X_I)$
 - 23: $H_L \leftarrow \mathcal{T}(H_0)$
 - 24: $\hat{y}_I \leftarrow \text{OutputLayer}(H_L[-1])$
 - 25: $\Delta \hat{t}_{I+1}, \hat{r}_{I+1}, \hat{e}_{I+1} \leftarrow \hat{y}_I$
 - 26: Compute $\hat{t}_{I+1} = t_I + \Delta \hat{t}_{I+1}$
 - 27: **return** $\hat{T} = (\hat{t}_{I+1}, \hat{r}_{I+1}, \hat{e}_{I+1})$
-

Algorithm 2 will accomplish one task prediction for the edge server with a time complexity of $O(ENL(I^2d + Id^2))$, the key influencing factors are sequence length I , model depth L , hidden dimension d , and the number of training rounds E ; among them, the sequence length I has the most significant influence on the computation, so the time complexity is $O(I^2)$.

Above we have analysed the two components of task offloading and resource allocation and proposed corresponding solutions.

C. Total Interaction

After the edge servers complete the prediction and evaluation, they will send the application demand to the cloud

servers, and multiple edge servers then use multiple rounds of Vickrey auctions to decide the cloud resource prices [38]. Vickrey auction is a sealed bid auction mechanism in which all participants submit their bids simultaneously, and the highest bidder wins the item but pays the price of the second-highest bid. This method effectively incentivizes bidders to truthfully reveal their genuine valuation of the item while avoiding strategic underbidding, thereby making the auction process simpler and more transparent and achieving efficient allocation of resources, which in turn reduces the psychological burden of worrying about overpaying and contributes to a fairer and more reasonable outcome. We update each round of edge server auction price with the following formula:

$$p_{n,m}^i = p_{win}^{i-1} + \gamma(p_{win}^{i-1} - p_{n,m}^{i-1}) \quad (18)$$

p_{win} is the $i - 1$ st round winning price. Thus, the overall algorithmic flow is as follows:

Algorithm 3 Transformer-Based Task Offloading and Resource Scheduling(TORS)

```

1: Input: edge set  $E_M$ , unit cost set  $h_M$ , location set  $L_M$ ,
   Amount of resources  $res_M$ , user set  $U_N$ , user location
    $L_N$ , user direction  $Dir_N$ , speed  $v_N$ , time  $T$ 
2: Output:  $\sum \Phi_{users}$ ,  $\sum \Phi_{edges}$ ,  $\sum \Phi_{cloud}$ ,  $F$ ,  $\sum$  QoS
3: Initialize  $N_{total} = 0$ ,  $N_{success} = 0$ 
4: while time  $< T$  and tasks arrive do
5:   Real-time updating of  $res_M$  and  $L_N$ 
6:    $N_{total} = N_{total} + 1$ 
7:   Calculate offloading results using Algorithm 1
8:   if  $E_t \neq \emptyset$  then
9:      $N_{success} = N_{success} + 1$ 
10:  end if
11:  for  $e_m \in E_t$  do
12:    Evaluate load of  $e_m$  (Algorithm 2)
13:    if load exceeds threshold then
14:      Add  $e_m$  to  $E_v$ 
15:    end if
16:  end for
17:   $E'_v = E_v$ 
18:  while  $E_v \neq \emptyset$  do
19:    Perform Vickrey auction
20:    Winner  $e_m$  exits  $E_v$ 
21:    Update  $p_{m,cloud}$  using Eq. (18)
22:  end while
23:  Update  $e_m$  resources  $res_M$  in  $E'_v$ 
24:  compute  $\Phi_{users}$ ,  $\Phi_{edges}$ ,  $\Phi_{cloud}$  using Eq. (6)(11)(14)
25:  compute  $QoS_{n,m}$  using Eq. (5)
26: end while
27: compute  $\sum \Phi_{users}$ ,  $\sum \Phi_{edges}$ ,  $\sum \Phi_{cloud}$ ,  $\sum$  QoS
28:  $F = N_{success}/N_{total}$ 
29: Return:  $\sum \Phi_{users}$ ,  $\sum \Phi_{edges}$ ,  $\sum \Phi_{cloud}$ ,  $F$ ,  $\sum$  QoS

```

In a single system interaction for task offloading, the time complexity is mainly affected by the number of edge servers, and the length of the sequence when training the model, so that the time complexity of the algorithm 3 is $O(M + MI^2)$, Algorithm 3 will return the total revenue and the overall task offload rate of the user, the edge server, and the cloud server

for a certain period of time. We aim to maximise the long-term utility.

V. EXPERIMENT

In the Edge-Cloud Computing system, the core elements include the revenue of all parties in the cloud-side-end, the total system revenue and the task offload rate, and we will evaluate the performance of the algorithms in terms of these main aspects. Also, we consider the user's QoS, which will also be an important element that we will evaluate.

A. Experimental Setup

The experiment includes multiple users with different initial locations, initial speeds, and directions of movement, who will randomly generate offloading tasks at different times; edge servers with different initial locations, different amounts of resources, and different unit costs, who come from different providers, so the market relationship is a non-cooperative game; and a cloud server far away from the city with a large amount of lower unit-cost computational resources, but with a high latency. Users will have two options of offloading tasks to a cloud server or a set of edge servers depending on their latency requirements. The initial positions of users and edge servers and the generation time of random tasks in the experiments obey normal distribution, and the parameters such as the amount of edge resources obey uniform distribution. We will show the specific experimental parameters in Table II. We generate many sets of different task instances and analyze the results by calculating the mean values using different algorithms.

TABLE II: Experimental parameter table

Parameter	Value
Bandwidth (B)	1 MHz
Initial Signal-to-Noise Ratio (SNR)	100 dB
attenuation rate	[0.001dB, 0.1dB]
Edge Server Resource (R_m)	[50G, 100G]
Task Package Size (D_n)	[10M, 20M]
Task Resource Demand (R_n)	[20G, 80G]
Number of Edge Servers (M)	[100, 300]
Number of Users (N)	[100, 500]
Time Horizon (T)	[200, 500]
User Mobility Speed (v_n)	[0 m/s, 15 m/s]

B. Comparison Results

We compared some of the more commonly used and classic algorithms and programs, and at the same time, divided the comparison results into two parts. The first part focuses on comparing the performance between the algorithms, i.e., the impact on each of the main elements and the experimental results; the second part focuses on comparing the effectiveness of the different algorithms as predictive models, using the impact on the core elements and the experimental results to assess the effectiveness of each algorithm as a predictive model.

1) *Effectiveness of the overall algorithm:* In order to verify the effectiveness of the proposed algorithms. We compared the heuristic greedy algorithm based on the competitiveness model, the multi-round vickrey auction algorithm based on the competitiveness model and the K-means clustering algorithm based on distance and price, respectively. The experimental results show the superiority of our algorithms.

- The first comparison algorithm is the heuristic greedy algorithm GCA based on the competitiveness model [39]. In order to reflect the fairness of the evaluation results, we integrate this algorithm into our experimental scenario. In the non-cooperative game session of edge servers, each edge server greedily strives for the highest allocation percentage, so it adjusts its own price to reach the highest level of competitiveness with other servers to maximize the revenue.
- The second comparison algorithm is the multi-round vickrey auction algorithm MVAC based on the competitiveness model [40], where the edge servers win the game based on their own competitiveness, maximize their allocation ratio and thus maximize the revenue. As a result, users will get higher QoS, while ensuring efficient resource utilization and balanced system revenue for all parties in the system.
- The third comparison algorithm is the distance and price based K-means clustering algorithm DPKA [41]. K-means categorizes the gaming servers into three groups based on different prices and different distances to the mobile users as high competitiveness edge servers, medium competitiveness edge servers and low competitiveness edge servers. Mobile users offload tasks to high competitiveness edge servers to get higher QoS.

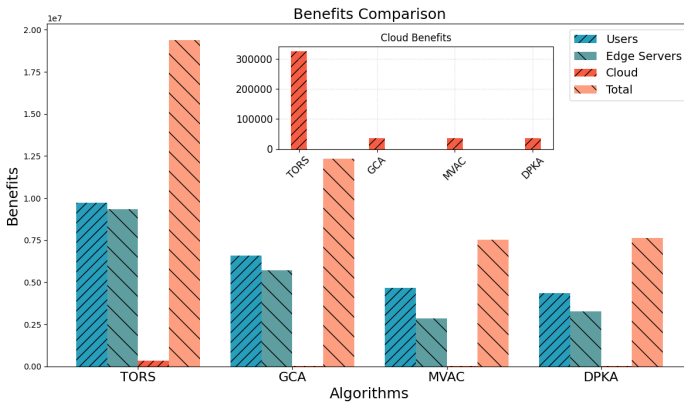


Fig. 2: Benefits comparison across algorithms

It can be observed from Figure 2 that TORS significantly outperforms the competing algorithms GCA, MVA, and DPKA in terms of total system benefits. Specifically, TORS improves the total system benefits by 57.17%, 157.26%, and 153.93%, respectively, when compared to GCA, MVA, and DPKA. This substantial improvement is attributed to TORS's ability to efficiently allocate resources and optimize task offloading, even under high-demand scenarios. Notably, while the overall system benefits show a significant rise, the cloud benefits

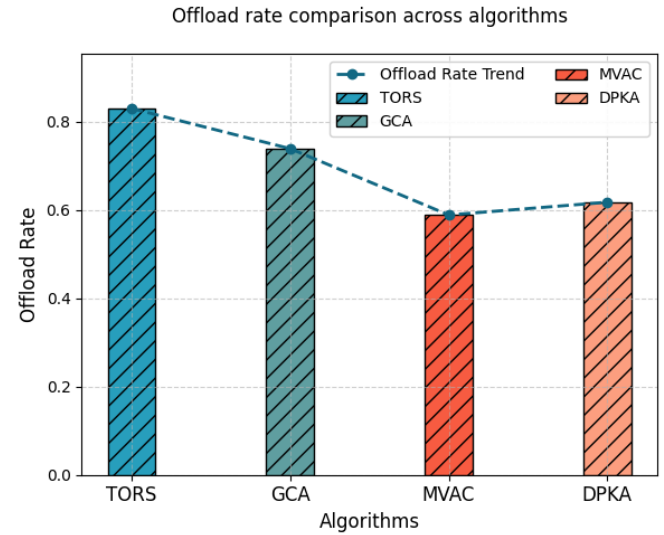


Fig. 3: Offload rate comparison across algorithms

are relatively lower due to the high number of mobile users and vehicles, which impose stringent latency requirements on task completion. This necessitates a shift in computation towards edge servers to meet these latency constraints, thereby reducing the cloud's direct contribution to the system benefits. To better illustrate these differences, the cloud benefits are depicted in dedicated subgraphs, enabling clearer visualization of their relative contributions. Furthermore, it is evident from the figure that the total benefits derived from edge servers, cloud servers, and users collectively are markedly greater under TORS than those achieved by the other algorithms. This demonstrates TORS's superior capability to balance computational loads and optimize task distribution across the system.

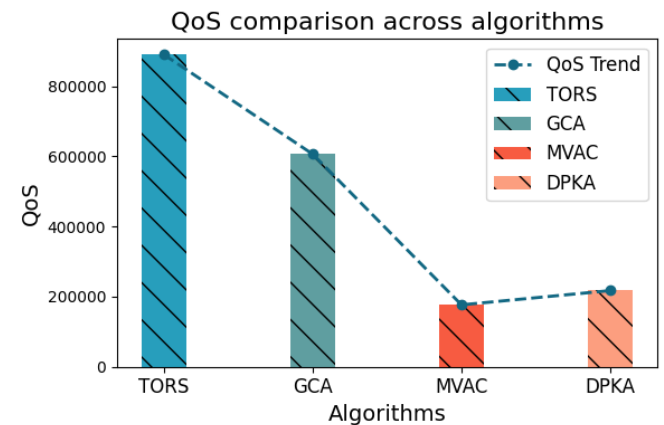


Fig. 4: QoS comparison across algorithms

Figure 3 illustrates the overall task offloading rates achieved by the four algorithms, highlighting TORS as the most effective. The offloading rate of TORS surpasses GCA, MVA, and DPKA by 12.32%, 40.96%, and 34.38%, respectively. A higher task offloading rate directly translates to a greater number of completed tasks within the system. This accom-

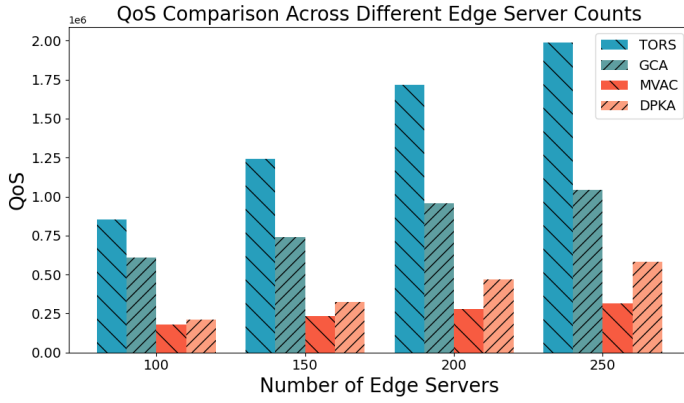


Fig. 5: Comparison of each algorithm with different number of edge servers

plishment has a twofold benefit: first, it optimizes resource utilization across the system, enhancing total system revenue; and second, it significantly improves the QoS for users by reducing delays and ensuring task completion within acceptable timeframes. Moreover, Figure 4 emphasizes the notable improvement in users' QoS achieved by TORS. With its efficient task allocation and offloading strategy, TORS facilitates the completion of a significantly higher number of tasks compared to the other algorithms. This directly contributes to a smoother user experience, reduced latency, and a more responsive system, underscoring the practicality and robustness of TORS in handling diverse and dynamic computational demands.

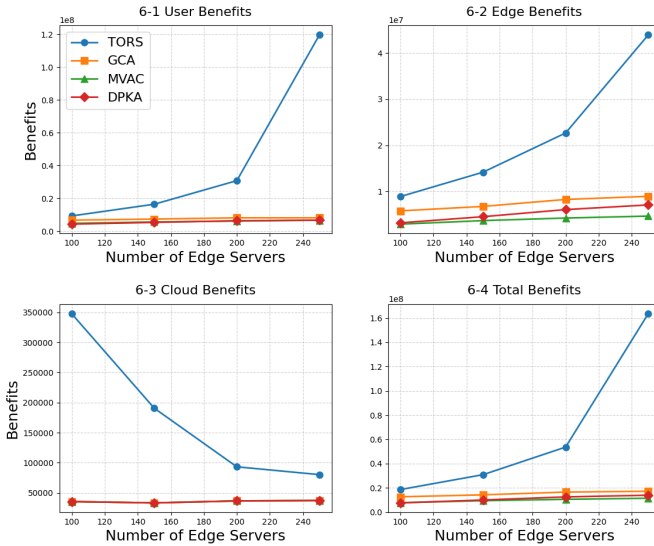


Fig. 6: System benefits for each algorithm with different number of edge servers.

Due to the specificity of the non-cooperative game with edge servers, we investigate the situation under different numbers of edge servers. It is obvious that due to the increase in the number of edge servers, the overall load capacity of the system rises, so more tasks can be offloaded, the task offloading rate

will be much higher, and the user will have a higher QoS. It is obvious from Fig. 5 that each algorithm positively affects the overall QoS of the user as the number of edge servers increases. The more the number of edge servers, the higher the QoS of the user.

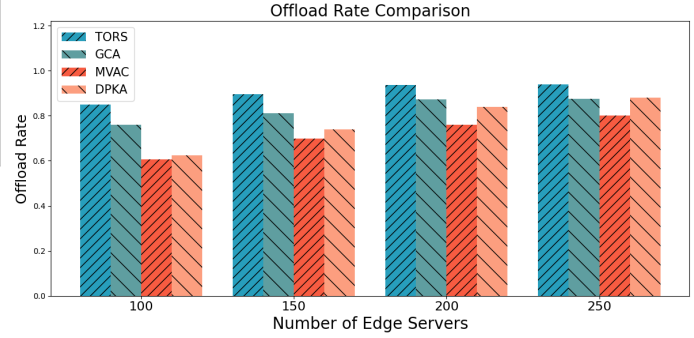


Fig. 7: Task offload rate for each algorithm with different number of edge servers.

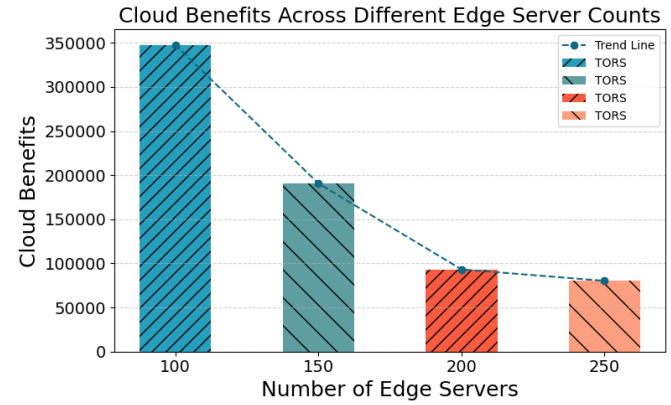


Fig. 8: Cloud benefits with different edge servers

We can also clearly draw the above conclusion from Fig. 6 and Fig. 7. As the number of edge servers increases, the overall benefit of each algorithm goes up and the offloading rate similarly all goes up together. Due to the increase in the number of edge servers, the number of offloading tasks increases and the increase in the total benefit of edge servers under each algorithm is significant. Moreover, we can see that TORS has excellent performance in different number of edge server scenarios and is higher than GCA, MVA and DPKA both in terms of total benefit of cloud servers, edge servers, users and overall task offloading rate.

It is important to note in particular that the overall benefit of cloud servers under TORS decreases as the number of edge servers increases, which we show in detail in Fig. 8. The decreasing trend of cloud server benefits under the increase of edge servers can be clearly seen. This is due to the fact that more edge servers have more resources and higher load capacity, and with sufficient resources of their own, the edge servers do not pre-request resources from the cloud, so the benefits of the cloud servers decline instead.

2) *Effectiveness of Prediction models*: We have designed a transformer-based prediction model. In practice, deep reinforcement learning is growing rapidly, and there are countless algorithms that can be used to perform task prediction. In order to verify the effectiveness and superiority of our exit algorithm, we compare it with several classical and commonly used algorithmic models. Due to the complexity of task prediction, we combine the task poke of the task and the resources and execution time required by the task for the prediction of the two models to reduce the interference of multiple data on the prediction results and thus improve the effectiveness of the compared algorithmic models.

- The first comparison algorithm is the baseline, i.e., the utility of the system without the use of a predictive model. It is based on the results derived from our proposed competitiveness-based resource pricing and task allocation.
- The second comparison algorithm is a task prediction algorithm based on a combination of linear regression and moving average (LINE-MOV) [42]. In statistics, linear regression is a type of regression analysis that models the relationship between one or more independent and dependent variables using a least-square function called a linear regression equation. It is a classical fitting algorithm with good predictive results for time-series type data; moving average is a popular technical analysis tool. If you get a set of measured values in sequence, take a certain number of data in order and calculate the arithmetic mean of all of them, the data obtained is called the moving average, and this method also has a good effect in predicting time series data. The combination of these two algorithms can work well for task prediction.
- The third comparison algorithm is based on the classical Long Short Term Memory Network LSTM and Random Forest (LSTM-RAN) [43]. Long Short-Term Memory Network is a temporal recurrent neural network, which is specially designed to solve the problem of long-term dependency that exists in general RNNs, all of which have a chained form of repeating neural network modules. In standard RNNs, this repeating structural module has only a very simple structure, such as a tanh layer. A random forest is a classifier that contains many decision trees and can be used to deal with both classification and regression problems as well as being suitable for dimensionality reduction problems. It is also very tolerant of outliers and noise, and has better prediction and classification performance compared to decision trees. The combination of these two algorithms is also good for task prediction.

From Fig. 9, it can be seen that the transformer-based prediction model has higher user and cloud server benefits compared to the other algorithms, which means that the transformer-based prediction model predicts the user's tasks more accurately, carries out more frequent scheduling of the edge cloud resources, and therefore accomplishes more user tasks thus increasing the total system benefits. In terms of edge server benefits, there is not much difference between the four

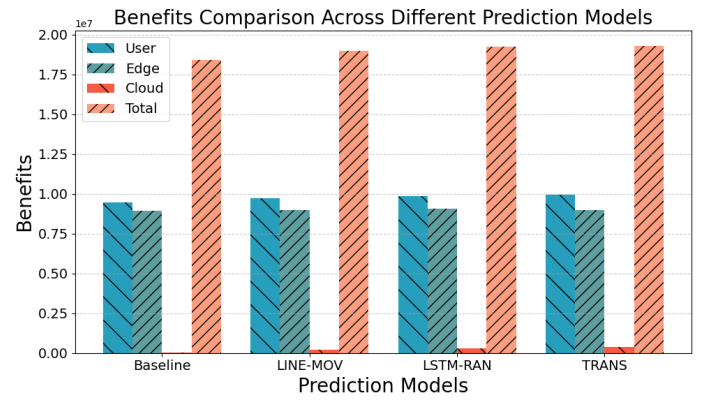


Fig. 9: Comparison of benefits under different prediction models

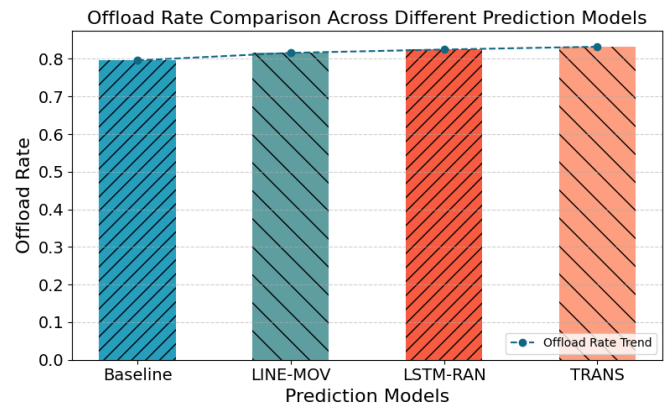


Fig. 10: Comparison of task offloading rates under different prediction models

prediction models, so the transformer-based prediction model is better in terms of total system benefits.

As can be seen from Fig. 10, the transformer-based prediction model has a higher task offloading rate, which is 3.63%, 1.63%, and 0.75% higher than Baseline, LINE-MOV, and LSTM-RAN, respectively, due to more frequent and accurate scheduling of the edge cloud resources and the edge servers have a higher load.

In this paper, by directly comparing the prediction accuracies, we demonstrate more intuitively the differences between different prediction models and provide a basis for selecting the most suitable model for specific application scenarios. Since Baseline does not use a prediction model, we compares the effectiveness of the other three prediction models. The comparison results in Fig. 11 show that the model based on the Transformer architecture has a significant advantage in prediction accuracy, which is a key factor in its ability to achieve high revenue and high task offload rate in mobile edge computing systems. This is also the reason why we use it. The prediction accuracy is defined as follows: after each edge server completes a prediction task using its personalized trained model, if a real task is offloaded and executed within the time window corresponding to the prediction result, the

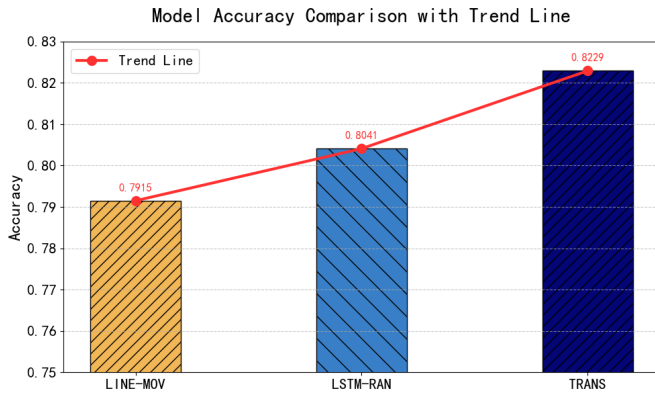


Fig. 11: Comparison of task offloading rates under different prediction models

prediction is considered successful. The prediction accuracy is then calculated as the ratio of the number of successful predictions to the total number of prediction tasks.

VI. CONCLUSION

In this study, we explore the pressing challenges facing cloud-edge-end computing networks, including high task offloading requirements for edge servers, competition among multiple edge service providers, and inefficiencies in resource pricing and allocation. Our proposed solution combines a competitiveness model for efficient edge task allocation and a transformer-based task prediction model to optimise task offloading and resource scheduling.

By structuring the system interaction into two distinct steps, we simplify the problem solving process and maximise the benefits for all stakeholders including edge service providers, cloud providers and end users. Results show that our approach improves task offloading efficiency, optimises resource utilisation and creates a competitive and transparent resource pricing environment. The solution lays the foundation for a more coordinated and efficient cloud-edge-end system, paving the way for sustainable and equitable growth of future network infrastructure.

Despite some progress in task offloading and resource scheduling, the reliance of current schemes on global offloading is not always effective in real-world applications, and issues related to partial offloading are still not adequately addressed. Future research should explore the integration of partial offloading mechanisms into existing frameworks, and combine the strengths of both global and local approaches with current approaches based on edge contention and task prediction to provide new theoretical insights for efficiently handling the various tasks in edge computing and IoT scenarios by providing new theoretical insights and technical support.

REFERENCES

- [1] F. J. Haro-Olmo, J. A. Alvarez-Bermejo, A. J. Varela-Vaca, and J. A. López-Ramos, "Blockchain-based federation of wireless sensor nodes," *The Journal of Supercomputing*, pp. 1–13, 2021. [I](#)
- [2] L. Barolli, F. Hussain, and M. Takizawa, "Special issue on intelligent edge, fog, cloud and internet of things (iot)-based services," *Computing*, 2021. [I](#)

- [3] Y. Ai, M. Peng, and K. Zhang, "Edge computing technologies for internet of things: a primer," *Digital Communications and Networks*, vol. 4, no. 002, pp. 77–86, 2018. [I](#)
- [4] M. Feng, W. Zhang, and M. Krunz, "Task partitioning and user association for latency minimization in mobile edge computing networks," *IEEE Transactions on Vehicular Technology*, 2021. [I](#)
- [5] G. Qu, H. Wu, R. Li, and P. Jiao, "Dmro: A deep meta reinforcement learning-based task offloading framework for edge-cloud computing," *IEEE Transactions on Network and Service Management*, vol. 18, no. 3, pp. 3448–3459, 2021. [I](#), [II](#)
- [6] T. Iggena, E. Ilyas, M. Fischer, R. Tnjes, and S. Christophersen, "Iotcrawler: Challenges and solutions for searching the internet of things," *Sensors*, vol. 21, no. 5, p. 1559, 2021. [II](#)
- [7] S. Shahhosseini, A. Anzanpour, I. Azimi, S. Labbaf, and A. M. Rahmani, "Exploring computation offloading in iot systems," *Information Systems*, 2021. [II](#)
- [8] A. Lakshmi and M. Vijayalakshmi, "An adaptive multi-cloud offloading using hierarchical game-theoretic approach," *International Journal of Intelligent Networks*, 2021. [II](#)
- [9] R. P. Mathur and M. Sharma, "A machine learning approach for offload decision making in mobile cloud computing," in *2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)*, 2021. [II](#)
- [10] F. Zhang, M. M. Wang, L. Shan, M. Zhang, and C. Xu, "Multi-user stochastic game for utility optimization in mobile ad hoc cloud," in *2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*, 2021, pp. 1–5. [II](#)
- [11] F. Hoseiny, S. Azizi, M. Shojafar, F. Ahmadiazar, and R. Tafazolli, "Pga: A priority-aware genetic algorithm for task scheduling in heterogeneous fog-cloud computing," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2021, pp. 1–6. [II](#)
- [12] Z. Sadreddini and H. Yanikomeroglu, "A novel centralized cloud-based mobile data rollover management," *IEEE Wireless Communications*, vol. PP, no. 99, pp. 1–6, 2021. [II](#)
- [13] H. He, W. Zhao, S. Huang, G. C. Fox, and Q. Wang, "Research on the architecture and its implementation for instrumentation and measurement cloud," *IEEE Transactions on Services Computing*, vol. 13, no. 5, pp. 944–957, 2020. [II](#)
- [14] T. Liu, S. Ni, X. Li, Y. Zhu, L. Kong, and Y. Yang, "Deep reinforcement learning based approach for online service placement and computation resource allocation in edge computing," *IEEE Transactions on Mobile Computing*, pp. 1–1, 2022. [II](#)
- [15] S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. Shen, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Transactions on Mobile Computing*, vol. 20, no. 3, pp. 939–951, 2021. [II](#)
- [16] L. Yang, S. Guo, L. Yi, Q. Wang, and Y. Yang, "Noscm: A novel offloading strategy for noma-enabled hierarchical small cell mobile-edge computing," *IEEE Internet of Things Journal*, vol. 8, no. 10, pp. 8107–8118, 2021. [II](#)
- [17] H. Gedawy, K. Habak, K. A. Harras, and M. Hamdi, "Ramos: A resource-aware multi-objective system for edge computing," *IEEE Transactions on Mobile Computing*, vol. 20, no. 8, pp. 2654–2670, 2021. [II](#)
- [18] S. Bi, L. Huang, H. Wang, and Y.-J. A. Zhang, "Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks," *IEEE Transactions on Wireless Communications*, vol. 20, no. 11, pp. 7519–7537, 2021. [II](#)
- [19] Z. Ma, S. Zhang, Z. Chen, T. Han, Z. Qian, M. Xiao, N. Chen, J. Wu, and S. Lu, "Towards revenue-driven multi-user online task offloading in edge computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 5, pp. 1185–1198, 2022. [II](#)
- [20] S. Yang, F. Li, S. Trajanovski, X. Chen, Y. Wang, and X. Fu, "Delay-aware virtual network function placement and routing in edge clouds," *IEEE Transactions on Mobile Computing*, vol. 20, no. 2, pp. 445–459, 2021. [II](#), [III-B](#)
- [21] X. Chen, J. Zhang, B. Lin, Z. Chen, K. Wolter, and G. Min, "Energy-efficient offloading for dnn-based smart iot systems in cloud-edge environments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 3, pp. 683–697, 2022. [II](#)
- [22] X. Ma, S. Wang, S. Zhang, P. Yang, C. Lin, and X. Shen, "Cost-efficient resource provisioning for dynamic requests in cloud assisted mobile edge computing," *IEEE Transactions on Cloud Computing*, vol. 9, no. 3, pp. 968–980, 2021. [II](#)
- [23] H. Yuan and M. Zhou, "Profit-maximized collaborative computation offloading and resource allocation in distributed cloud and edge computing

- systems,” *IEEE Transactions on Automation Science and Engineering*, vol. 18, no. 3, pp. 1277–1287, 2021. [II](#)
- [24] S. Tian, C. Chi, S. Long, S. Oh, Z. Li, and J. Long, “User preference-based hierarchical offloading for collaborative cloud-edge computing,” *IEEE Transactions on Services Computing*, pp. 1–1, 2021. [II](#)
- [25] Y. Miao, G. Wu, M. Li, A. Ghoneim, M. Al-Rakhamei, and M. S. Hos-sain, “Intelligent task prediction and computation offloading based on mobile-edge cloud computing,” *Future Generation Computer Systems*, vol. 102, pp. 925–931, 2020. [II](#)
- [26] G. Wu, Y. Miao, Y. Zhang, and A. Barnawi, “Energy efficient for uav-enabled mobile edge computing networks: Intelligent task prediction and offloading,” *Computer Communications*, vol. 150, pp. 556–562, 2020. [II](#)
- [27] J. Chen and Y. Yang, “A context-aware approach to task scheduling for time series data prediction in mobile edge computing,” *Mobile Networks and Applications*, vol. 28, pp. 421–431, 2023. [II](#)
- [28] Y. Xiao, Y. Yao, K. Chen, W. Tang, and F. Zhu, “A simulation task partition method based on cloud computing resource prediction using ensemble learning,” *Simulation Modelling Practice and Theory*, vol. 119, p. 102595, 2022. [II](#)
- [29] X. Duan, F. Xu, and Y. Sun, “Research on offloading strategy in edge computing of internet of things,” in *2020 International Conference on Computer Network, Electronic and Automation (ICCNEA)*, 2020, pp. 206–210. [II](#)
- [30] Y. Ding, K. Li, C. Liu, and K. Li, “A potential game theoretic approach to computation offloading strategy optimization in end-edge-cloud computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 6, pp. 1503–1519, 2022. [II](#)
- [31] S. Long, W. Long, Z. Li, K. Li, Y. Xia, and Z. Tang, “A game-based approach for cost-aware task assignment with qos constraint in collaborative edge and cloud environments,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 7, pp. 1629–1640, 2021. [II](#), [III-B](#)
- [32] J. Li, W. Liang, W. Xu, Z. Xu, X. Jia, W. Zhou, and J. Zhao, “Maximizing user service satisfaction for delay-sensitive iot applications in edge computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 5, pp. 1199–1212, 2022. [II](#)
- [33] Z. Xiong, J. Kang, D. Niyato, P. Wang, and H. V. Poor, “Cloud/edge computing service management in blockchain networks: Multi-leader multi-follower game-based admm for pricing,” *IEEE Transactions on Services Computing*, vol. 13, no. 2, pp. 356–367, 2020. [II](#)
- [34] G. Cui, Q. He, X. Xia, P. Lai, F. Chen, T. Gu, and Y. Yang, “Interference-aware saas user allocation game for edge computing,” *IEEE Transactions on Cloud Computing*, pp. 1–1, 2020. [II](#)
- [35] Y. Han, D. Niyato, C. Leung, C. Miao, and D. I. Kim, “Dynamics in coded edge computing for iot: A fractional evolutionary game approach,” *IEEE Internet of Things Journal*, pp. 1–1, 2022. [II](#)
- [36] F. Zhang and M. M. Wang, “Stochastic congestion game for load balancing in mobile-edge computing,” *IEEE Internet of Things Journal*, vol. 8, no. 2, pp. 778–790, 2021. [II](#)
- [37] G. Sun, L. He, Z. Sun, Q. Wu, S. Liang, J. Li, D. Niyato, and V. C. Leung, “Joint task offloading and resource allocation in aerial-terrestrial uav networks with edge and fog computing for post-disaster rescue,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 9, pp. 8582–8600, 2024. [II](#)
- [38] X. Zhang, K. An, B. Zhang, Z. Chen, Y. Yan, and D. Guo, “Vickrey auction-based secondary relay selection in cognitive hybrid satellite-terrestrial overlay networks with non-orthogonal multiple access,” *IEEE Wireless Communications Letters*, vol. 9, no. 5, pp. 628–632, 2020. [IV-C](#)
- [39] M. K. Somesula, R. R. Rout, and D. V. Somayajulu, “Greedy cooperative cache placement for mobile edge networks with user preferences prediction and adaptive clustering,” *Ad Hoc Networks*, vol. 140, p. 103051, 2023. [V-B1](#)
- [40] Q. Wang, S. Guo, J. Liu, C. Pan, and L. Yang, “Profit maximization incentive mechanism for resource providers in mobile edge computing,” *IEEE Transactions on Services Computing*, vol. 15, no. 1, pp. 138–149, 2019. [V-B1](#)
- [41] C. Yang, X. Xu, M. Bilal, Y. Wen, and T. Huang, “Deep-deterministic-policy-gradient-based task offloading with optimized k-means in edge-computing-enabled iomt cyber-physical systems,” *IEEE Systems Journal*, 2023. [V-B1](#)
- [42] R. Naha, S. Garg, S. K. Battula, M. B. Amin, and D. Georgakopoulos, “Multiple linear regression-based energy-aware resource allocation in the fog computing environment,” *Computer Networks*, vol. 216, p. 109240, 2022. [V-B2](#)
- [43] S. Tuli, S. S. Gill, P. Garraghan, R. Buyya, G. Casale, and N. R. Jennings, “Start: Straggler prediction and mitigation for cloud computing

environments using encoder lstm networks,” *IEEE Transactions on Services Computing*, vol. 16, no. 1, pp. 615–627, 2021. [V-B2](#)



Shujuan Tian received the Ph.D. degree in Central South University in 2016. From September 2019 to September 2020, she was a postdoctoral researcher in the Computer Science department of Illinois Institute of Technology (IIT), and cooperated with Prof. Pengjun Wan. She is currently a Professor in the School of Computer Science & School of Cyberspace Security, Xiangtan University. Her research interests include Mobile Edge Computing, VANET, Internet of Things, Wireless Signal Control and Processing, etc.



Keke Xu is currently pursuing a master's degree in the School of Computer Science & School of Cyberspace Security at Xiangtan University in China. His current research interests include edge computing and resource scheduling.



Shuhuan Xiang is studying at Xiangtan University in China in 2023. He is pursuing a Master's degree in Computer Science and Technology from Xiangtan University in China in 2023. His current research interests include federated learning, Internet of vehicles, and machine learning.



Xingxia Dai received the B.S. degree from Xiangtan University, China, in 2018, and the Ph.D. degree from the College of Information Science and Electronic Engineering, Hunan University, Changsha, China, in 2024. She is currently a lecturer in the School of Computer Science & School of Cyberspace Security, Xiangtan University. Her current research interests include resource allocation and mobile edge computing.



Zhu Xiao received the M.S. and Ph.D. degrees in communication and information systems from Xidian University, Xi'an, China, in 2007 and 2009, respectively. From 2010 to 2012, he was a Research Fellow with the Department of Computer Science and Technology, University of Bedfordshire, Luton, U.K. He is currently a Professor with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. His research interests include machine learning, mobile computing, and intelligent information processing. He has actively served on many conference committees. He is an Associate Editor of the IEEE TITS.



Li Zeng is a Ph.D. She received a bachelor's degree in software engineering and a master's degree in computer science and technology from Xiangtan University in 2013 and 2016, respectively. Currently engaged in research in machine learning, cryptanalysis, bioinformatics, etc.